

Tema 0.

Repaso de Python para lingüistas

Primer encargo en LexiData: preparar textos reales para análisis lingüístico computacional

Prácticas | Programación para PLN | Universidad Pontificia Comillas

Proyecto de práctica

Encargo

LexiData ha recibido un pequeño encargo de una editorial que quiere explorar automáticamente comentarios de lectores. Antes de construir herramientas complejas, el equipo necesita un prototipo básico en Python que:

1. guarde comentarios como strings;
2. limpie puntuación y mayúsculas;
3. divida textos en palabras;
4. calcule longitudes;
5. cuente frecuencias;
6. detecte palabras de interés;
7. prepare resultados fáciles de revisar.

Tu papel es actuar como lingüista computacional junior. Debes escribir código claro, comentado y explicable para una persona no técnica.

Ejercicios de práctica

Ejercicio 1. Ficha de entrada del proyecto

Crea tres variables:

- `cliente`, con el nombre "Editorial Horizonte";
- `proyecto`, con el nombre "Análisis inicial de comentarios lectores";
- `responsable`, con tu nombre.

Muestra una ficha por pantalla usando `print()` y saltos de línea.

Pista: usa `\n`.

Ejercicio 2. Primer comentario como string

Guarda en una variable llamada `comentario_1` el siguiente texto:

Me encanto la novela, pero el final fue demasiado rápido.

Después:

8. imprime el comentario;
9. calcula cuantos caracteres tiene;
10. comprueba si la palabra "novela" aparece en el comentario.

Ejercicio 3. Comillas en una cita

La editorial quiere guardar esta cita:

La lectora escribió: "el personaje principal parece muy real".

Guárdala correctamente en una variable llamada `cita` y muéstrala por pantalla.

Ejercicio 4. Normalización básica

Guarda este comentario:

```
La NOVELA es interesante. La novela tiene ritmo.
```

Crea una nueva variable llamada `comentario_limpio` que contenga el texto en minúsculas y sin puntos.

Después imprime el resultado.

Ejercicio 5. Tokenización sencilla con ``split``

Usa el texto limpio del ejercicio anterior y crea una lista de palabras con `.split()`.

Después:

11. imprime la lista;
12. imprime cuantos tokens contiene;
13. imprime el primer token;
14. imprime los tres últimos tokens.

Ejercicio 6. Vocabulario único

A partir de la lista de tokens del ejercicio anterior, crea un conjunto llamado `vocabulario`.

Después:

15. imprime el conjunto;
16. imprime cuantas palabras únicas contiene;
17. explica en un comentario por qué el conjunto tiene menos elementos que la lista original.

Ejercicio 7. Diccionario de traducción editorial

Crea un diccionario llamado `glosario` con estas entradas:

- `"novela": "novel";`
- `"personaje": "character";`
- `"lector": "reader";`
- `"ritmo": "pace".`

Después:

18. consulta la traducción de `"ritmo"`;
19. comprueba si `"novela"` está en el diccionario;
20. comprueba si `"novel"` está en el diccionario y explica el resultado en un comentario.

Ejercicio 8. Clasificación de palabras por longitud

Crea una variable `palabra` con el valor `"caracterizacion"`.

Escribe un condicional:

- si la palabra tiene más de 10 caracteres, imprime `"palabra larga"`;
- si no, imprime `"palabra corta o media"`.

Ejercicio 9. Longitud media de palabras

Dado el texto:

```
texto = "la protagonista evoluciona durante toda la novela"
```

Calcula la longitud media de sus palabras usando:

- `.split()`;
- una variable acumuladora;
- un bucle `for`;
- `len()`.

Ejercicio 10. Frecuencias sin limpiar

Escribe una función llamada `frecuencia_texto(texto)` que devuelva un diccionario con la frecuencia de cada palabra usando `.split()`.

Pruébala con:

```
"la novela gusta la novela emociona"
```

Ejercicio 11. Frecuencias limpias

Escribe una función llamada `frecuencia_texto_limpia(texto)` que:

21. pase el texto a minúsculas;
22. elimine `.`, `,`, `&`, `?`, `¡`, `!`;
23. divida el texto en palabras;
24. devuelva un diccionario de frecuencias.

Pruébala con:

```
"La novela gusta, la novela emociona. ¿La recomiendas?"
```

Ejercicio 12. Palabras de interés editorial

La editorial quiere saber si un comentario menciona alguno de estos aspectos:

```
aspectos = ["ritmo", "personaje", "final", "estilo"]
```

Dado:

```
comentario = "El personaje principal funciona, pero el final es precipitado."
```

Limpia el comentario de forma básica y escribe un bucle que imprima los aspectos encontrados.

Ejercicio 13. Miniinforme automático

Crea una función llamada `informe_comentario(texto)` que devuelva un diccionario con:

- `"texto_limpio"`: texto en minúsculas y sin puntuación básica;
- `"tokens"`: lista de palabras;
- `"numero_tokens"`: cantidad total de tokens;
- `"vocabulario"`: conjunto de palabras únicas;
- `"numero_palabras_unicas"`: cantidad de palabras únicas;
- `"frecuencias"`: diccionario de frecuencias.

Pruebala con:

```
"La novela tiene buen ritmo. El ritmo ayuda a la lectura."
```

Ejercicio 14. Tokenización con NLTK

Si tienes `nltk` instalado, importa `TreebankWordTokenizer` y tokeniza:

```
"La novela tiene ritmo, estilo y personajes memorables."
```

Compara el resultado con `.split()`. Escribe un comentario breve sobre la diferencia.

Ejercicio 15. Revisión profesional del código

Revisa tu solución del ejercicio 13 y añade comentarios que expliquen:

- que hace cada bloque;
- por qué se limpia el texto;
- por qué se usa un diccionario para frecuencias;
- por qué se usa un conjunto para vocabulario.

El objetivo no es escribir más código, sino hacerlo comprensible para otra persona del equipo.

Criterios de evaluación

Criterio	Indicadores
Comprensión básica de Python	usa variables, strings, operadores y <code>print()</code> correctamente
Trabajo con texto	aplica <code>lower()</code> , <code>replace()</code> , <code>split()</code> y conteos sencillos
Colecciones	distingue lista, diccionario y conjunto
Control de flujo	usa <code>if</code> , <code>else</code> y <code>for</code> con indentación correcta
Funciones	encapsula procesos y devuelve resultados con <code>return</code>
Razonamiento lingüístico	entiende por qué limpiar, tokenizar y contar
Claridad profesional	escribe nombres de variables comprensibles y comentarios útiles